AWS Static Portfolio Deployment with CI/CD

End-to-End CI/CD Deployment using AWS & GitHub Actions

Shariqua Patel Farooque

AWS Static Portfolio Deployment with CI/CD

(GitHub Actions + S3 + CloudFront)

1. Project Overview

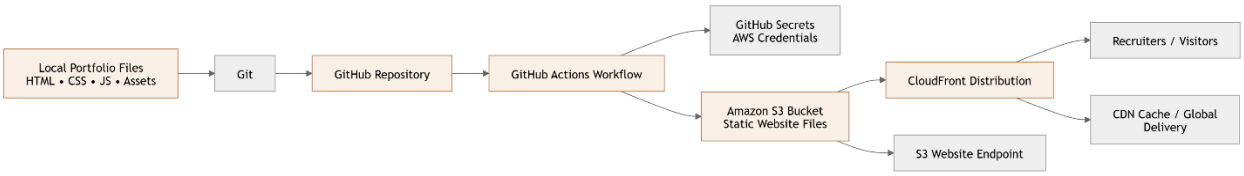
This project demonstrates a complete end-to-end deployment of a static portfolio website using AWS services and a CI/CD pipeline. The website is hosted on Amazon S3 and delivered globally using CloudFront.

The deployment process is fully automated using GitHub Actions. Whenever changes are pushed to the GitHub repository, the pipeline automatically updates the S3 bucket and reflects changes on the live website.

This project highlights practical knowledge of cloud hosting, automation, and deployment workflows.

2. Architecture Overview

Workflow:



Local Development → GitHub Repository → GitHub Actions (CI/CD) → Amazon S3 → CloudFront → End Users

The architecture ensures:

- Automated deployment
- Scalable hosting
- Fast global delivery

3. Tools and Technologies Used

- **AWS Services**
  - Amazon S3 (Static Website Hosting)
  - Amazon CloudFront (Content Delivery Network)
  - AWS IAM (User and Policy Management)
- **DevOps Tools**
  - Git & GitHub
  - GitHub Actions (CI/CD Pipeline)
- **Frontend**
  - HTML
  - CSS
  - JavaScript

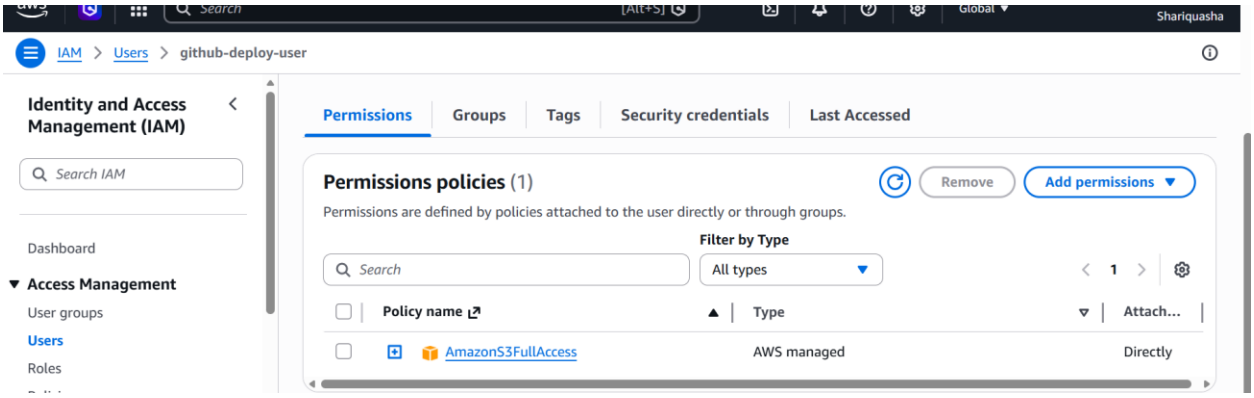
4. Project Workflow

1. Website files are developed locally (HTML, CSS, JS, assets).
2. Code is pushed to a GitHub repository.
3. GitHub Actions workflow is triggered automatically on push.
4. AWS credentials are securely accessed via GitHub Secrets.
5. The workflow deploys updated files to the S3 bucket.
6. CloudFront serves the updated content to users.
7. Cache invalidation is performed when required to reflect changes.

5. IAM Configuration

An IAM user was created specifically for deployment purposes.

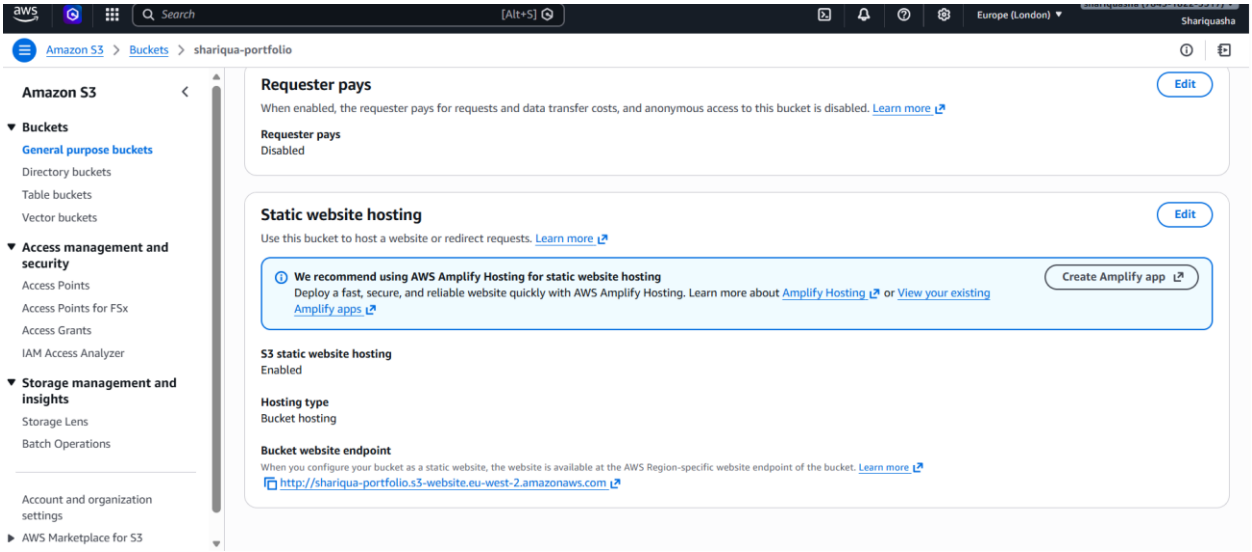
Custom policy attached with limited permissions:



AWS credentials (Access Key and Secret Key) were securely stored in GitHub Secrets and used by the CI/CD pipeline.

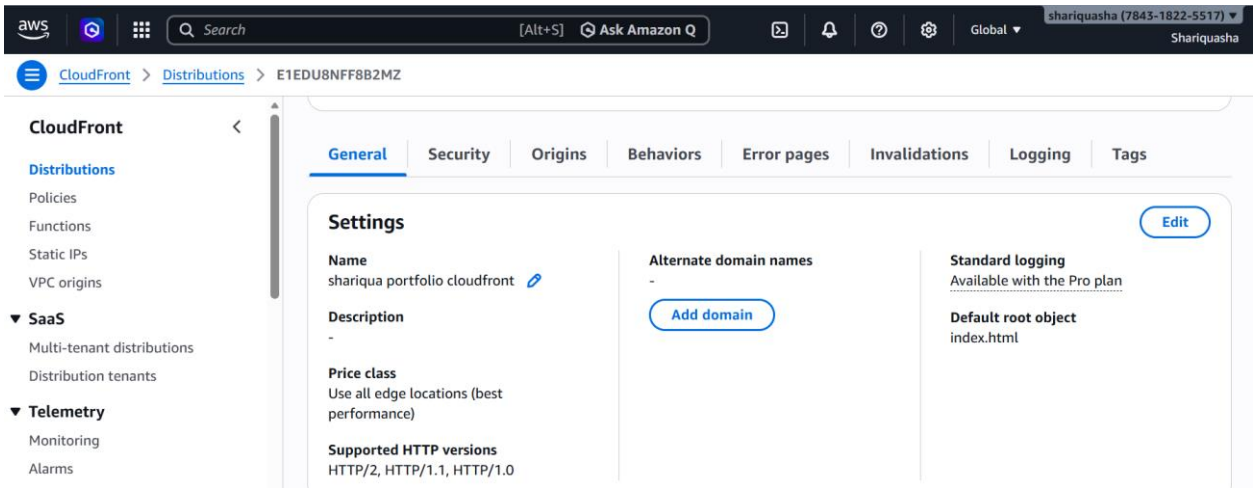
6. S3 Bucket Configuration

- S3 bucket created to host static website files
- Static website hosting enabled
- Index document set to index.html
- Public access configured using bucket policy
- Website accessible via S3 endpoint



7. CloudFront Configuration

- CloudFront distribution created
- Origin configured to S3 bucket
- Provides global CDN delivery
- Improves performance and availability
- Handles HTTPS and caching



8. CI/CD Pipeline (GitHub Actions)

A GitHub Actions workflow was configured to automate deployment.

Key Features:

- Trigger: On every push to main branch
- Uses AWS credentials from GitHub Secrets
- Deploys files using:
    - aws s3 sync
- Automatically updates the live website

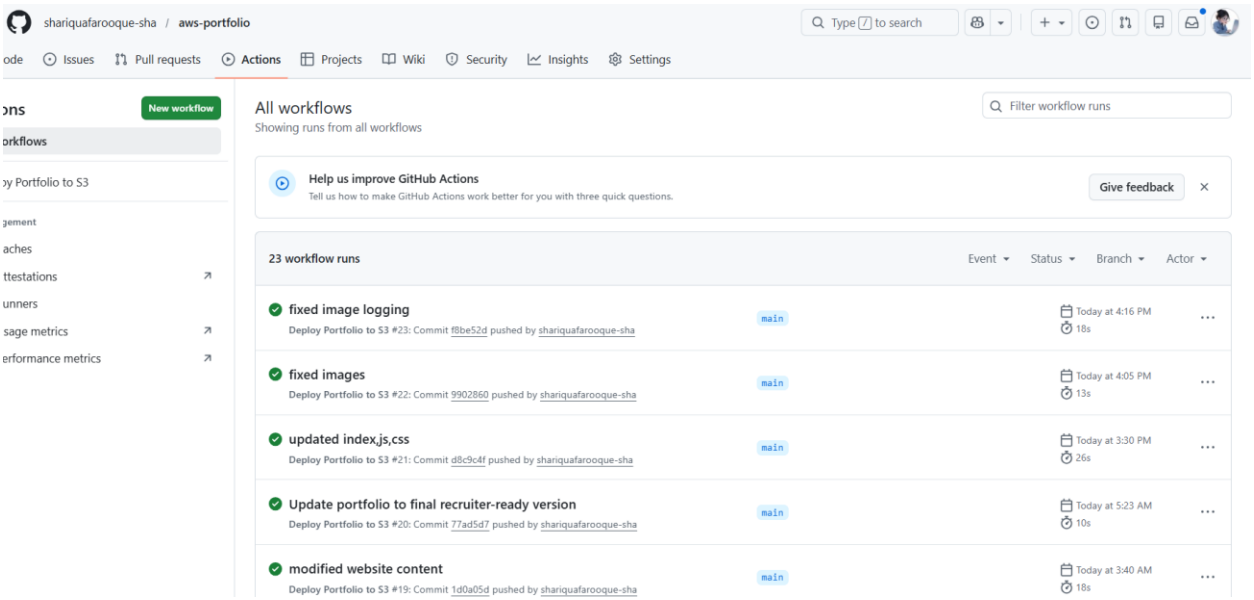
```
PS C:\Users\user\Documents\portfolio-aws> git add .
PS C:\Users\user\Documents\portfolio-aws> git commit -m " fixed image logging "
[main f8be52d]  fixed image logging
 1 file changed, 0 insertions(+), 0 deletions(-)
 rename assets/screenshots/s3_bucket/{bucket_ Logging.png => bucket_logging.png} (100%)
PS C:\Users\user\Documents\portfolio-aws> git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 424 bytes | 424.00 KiB/s, done.
Total 5 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/shariquafarooque-sha/aws-portfolio.git
   9902860..f8be52d  main -> main
PS C:\Users\user\Documents\portfolio-aws>
```

9. Screenshots and Implementation Proof

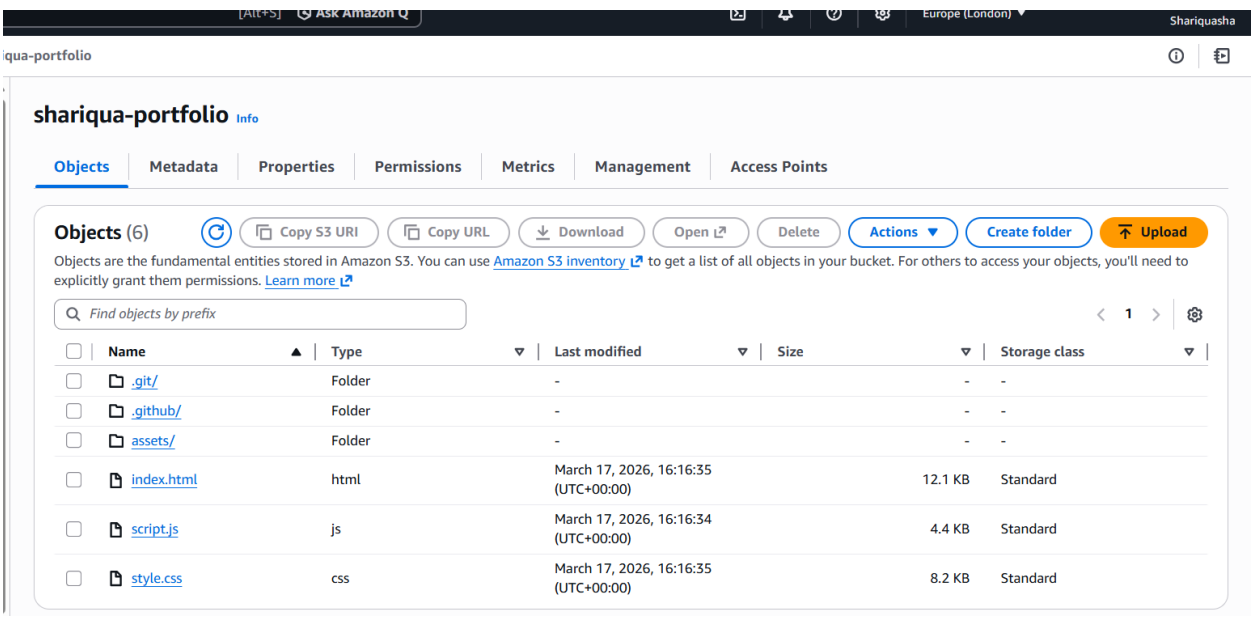
The project includes screenshots covering:

- IAM user and permissions
- GitHub Actions workflow and successful runs
- S3 bucket configuration and uploaded files
- CloudFront distribution setup
- Live website output (S3 and CloudFront)

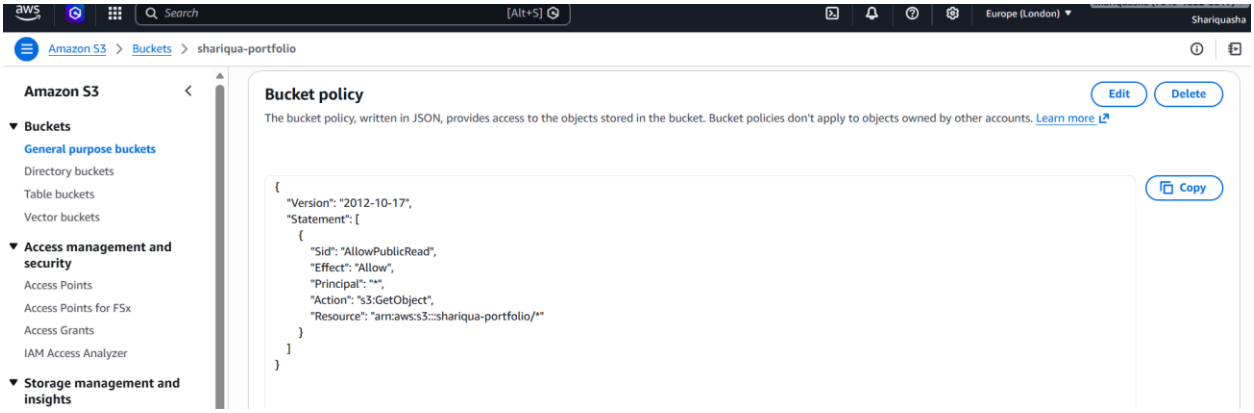
These screenshots provide proof of hands-on implementation.



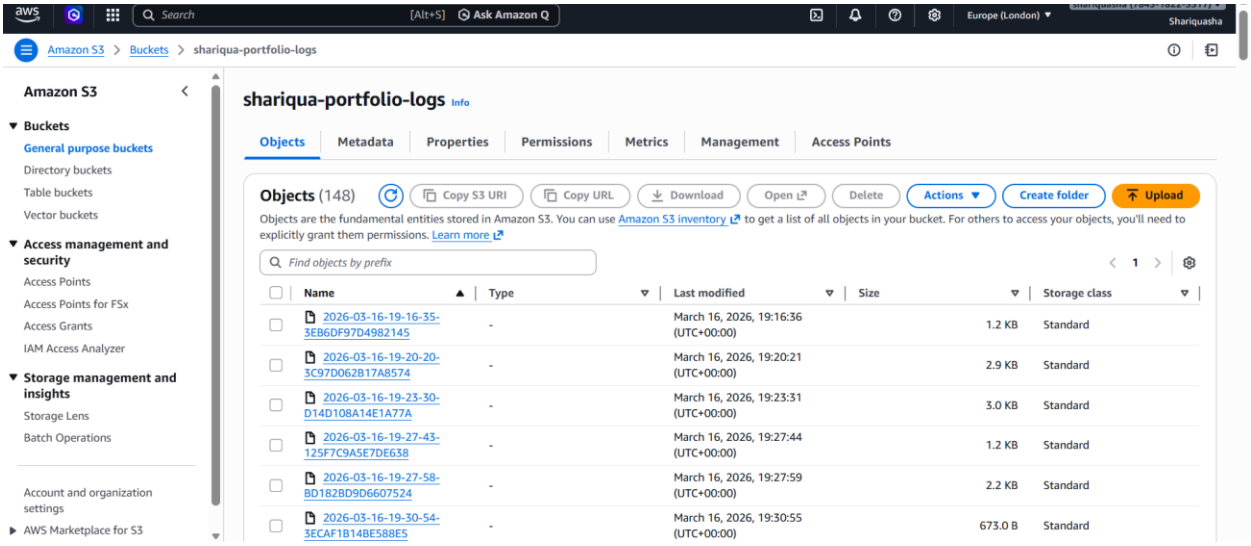
GitHub Actions workflow and successful runs



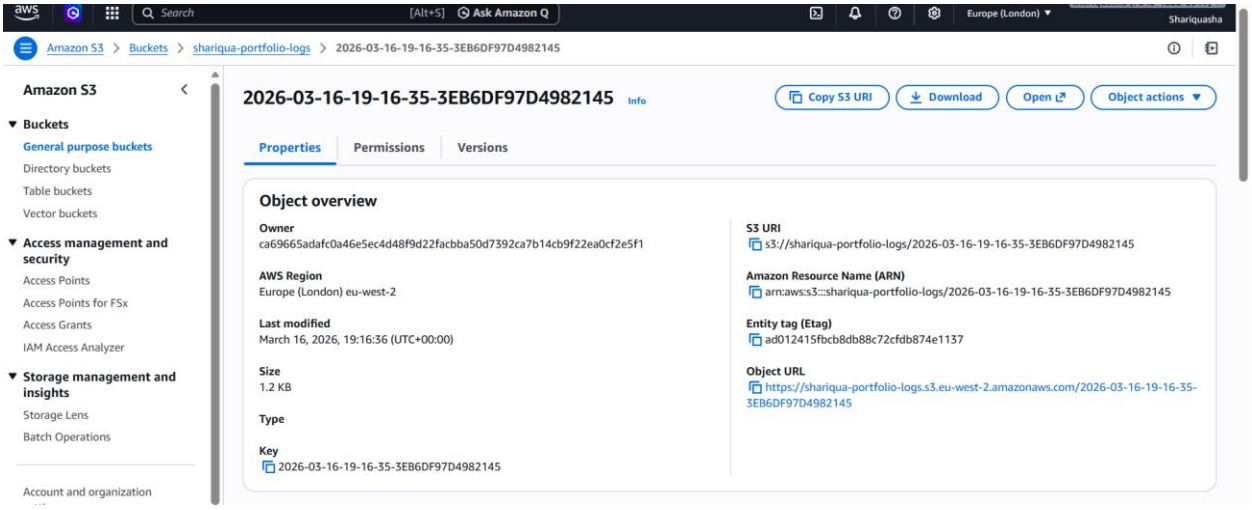
S3 bucket configuration and uploaded files



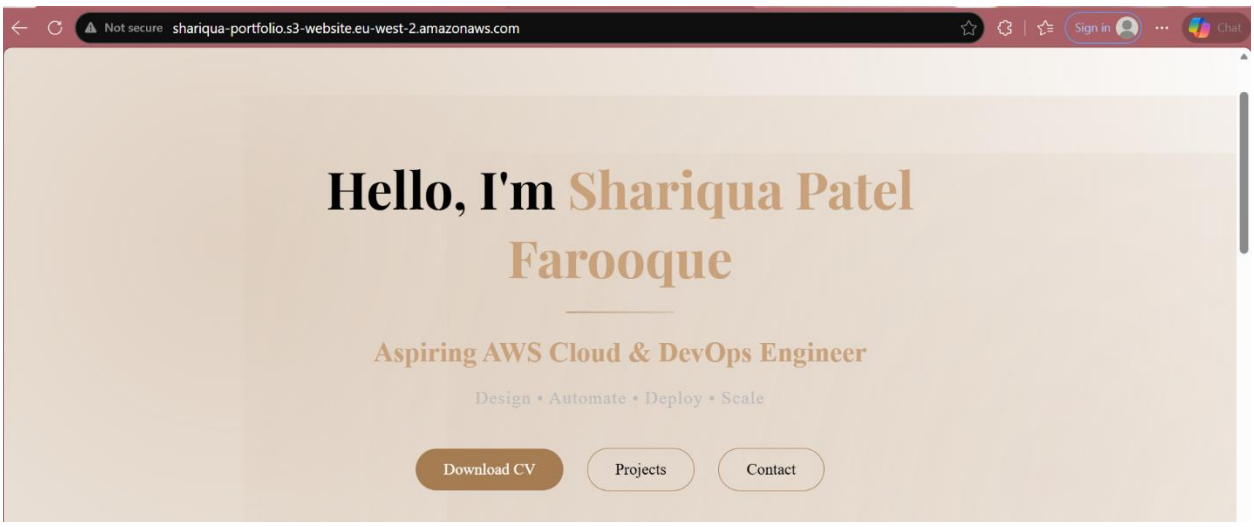
S3 Bucket Policy



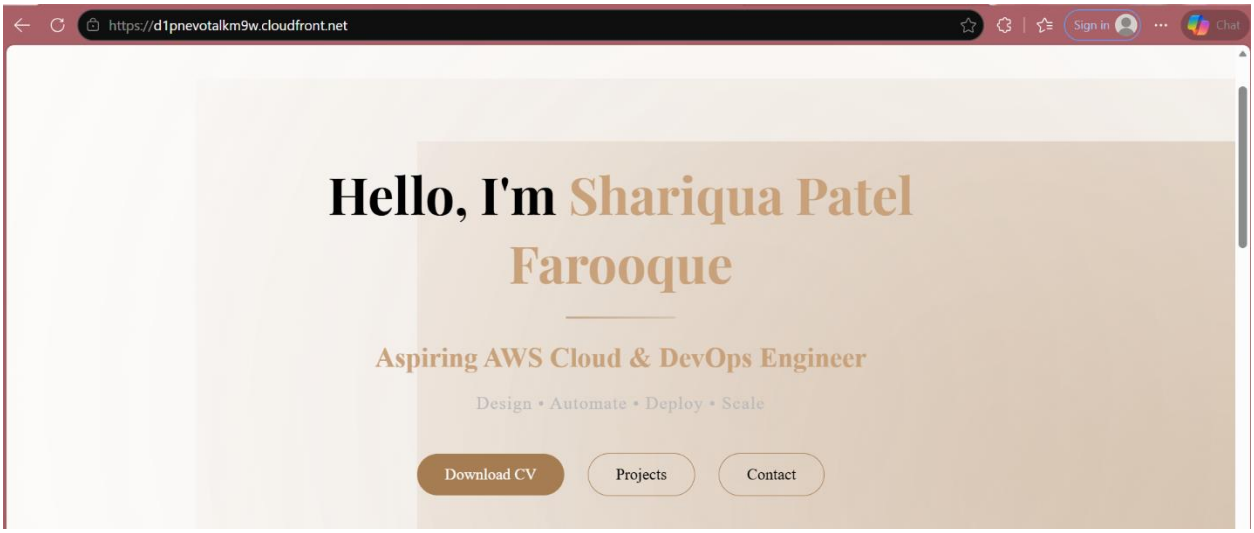
Logs Bucket List



Log File Details



S3 Live Website Output

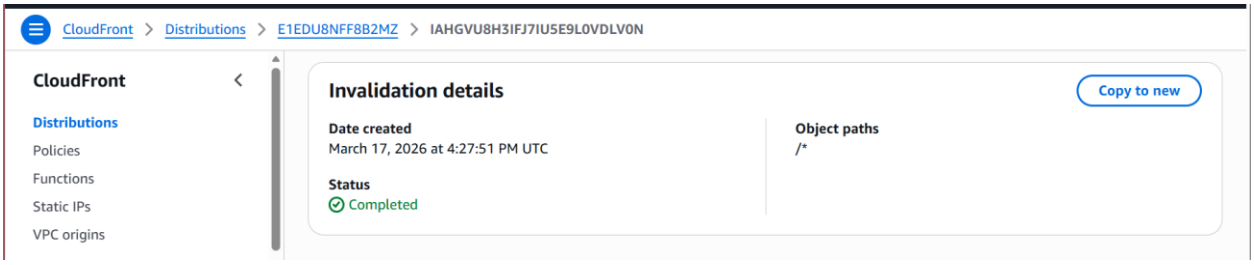


CloudFront Live Website Output

10. Challenges Faced

- During the implementation, the following challenges were encountered:
- CloudFront caching issues (old content not updating)
- Fixed using cache invalidation (/*)
- Broken images due to incorrect file paths
- Debugging folder structure and asset linking

These challenges helped improve troubleshooting and debugging skills.



CloudFront Invalidation

11. Key Learnings

- Understanding CI/CD pipeline workflow
- AWS S3 static hosting setup
- IAM permission management
- Secure handling of AWS credentials
- CloudFront caching and invalidation concepts
- End-to-end deployment automation

12. Future Improvements

- Replace IAM user with IAM role (OIDC-based authentication)
- Add custom domain with Route 53
- Enable HTTPS using AWS Certificate Manager (ACM)
- Implement monitoring using CloudWatch
- Add logging and alerting

S3 Website: http://shariqua-portfolio.s3-website.eu-west-2.amazonaws.com
CloudFront Website: https://d1pnevotalkm9w.cloudfront.net/

13. Conclusion

This project demonstrates a practical implementation of cloud deployment and CI/CD automation. It reflects the ability to build, deploy, and manage cloud-based applications using AWS and DevOps practices.

The workflow ensures efficiency, scalability, and automation, making it suitable for real-world applications.